

Phase 3: Scheduler & Lock-Free Concurrency - Implementation Guide

Overview

This document provides detailed implementation guidance for Phase 3 of the BDI kernel C23 refactor. It covers API usage, code examples, integration patterns, and best practices.

Table of Contents

- [1. Scheduler API](#)
- [2. Task Management API](#)
- [3. SMP API](#)
- [4. IPI API](#)
- [5. Integration Examples](#)
- [6. Performance Tuning](#)
- [7. Troubleshooting](#)

Scheduler API

Initialization

```
#include "scheduler.h"

// Initialize the scheduler (called once at boot)
void scheduler_init(void) {
    // Initializes global scheduler state
    // Sets up per-CPU run queues
    // Configures tickless timer
}
```

Core Scheduling Functions

schedule()

The main scheduling function that selects the next task to run.

```
// Schedule next task (called from timer interrupt or voluntary yield)
void schedule(void) {
    // 1. Save current task context
    // 2. Select next task from run queue
    // 3. Restore next task context
    // 4. Switch to next task
}
```

Usage Example:

```
// Voluntary yield from current task
void task_yield(void) {
    schedule(); // Give up CPU to another task
}

// Timer interrupt handler
void timer_interrupt_handler(void) {
    // Update tick count
    scheduler_tick();

    // Reschedule if time slice expired
    if (current_task_time_slice_expired()) {
        schedule();
    }
}
```

scheduler_tick()

Updates scheduler state on each timer tick (tickless mode adjusts frequency).

```
// Called from timer interrupt
void scheduler_tick(void) {
    // Update tick count atomically
    // Check for sleeping tasks to wake
    // Update current task time slice
    // Trigger reschedule if needed
}
```

Task State Queries

```
// Get current running task
struct task *get_current_task(void);

// Get number of running tasks
uint32_t get_num_running_tasks(void);

// Get scheduler statistics
struct scheduler_stats {
    uint64_t total_context_switches;
    uint64_t total_ticks;
    uint32_t num_tasks;
};

void get_scheduler_stats(struct scheduler_stats *stats);
```

Task Management API

Task Creation

```
#include "task.h"

// Create a new task
struct task *task_create(
    const char *name,           // Task name (for debugging)
    task_entry_t entry_point,   // Entry point function
    void *arg,                 // Argument to entry point
    uint32_t priority,         // Task priority (0-255)
    size_t stack_size          // Stack size in bytes
);
```

Usage Example:

```
// Task entry point
void my_task_entry(void *arg) {
    int *data = (int *)arg;

    while (1) {
        // Do work
        process_data(*data);

        // Voluntary yield
        task_yield();
    }
}

// Create and start task
void start_my_task(void) {
    int *data = malloc(sizeof(int));
    *data = 42;

    struct task *task = task_create(
        "my_task",
        my_task_entry,
        data,
        PRIORITY_NORMAL,
        8192 // 8KB stack
    );

    if (task) {
        task_start(task);
    }
}
```

Task Control

```
// Start a task (add to run queue)
int task_start(struct task *task);

// Stop a task (remove from run queue)
int task_stop(struct task *task);

// Destroy a task (cleanup resources)
void task_destroy(struct task *task);

// Yield CPU to another task
void task_yield(void);

// Sleep for specified milliseconds
void task_sleep(uint32_t milliseconds);

// Block task (waiting for event)
void task_block(struct task *task);

// Unblock task (event occurred)
void task_unblock(struct task *task);
```

Task State Transitions

```
// Task states
enum task_state {
    TASK_READY,      // Ready to run
    TASK_RUNNING,    // Currently running
    TASK_BLOCKED,    // Waiting for I/O or event
    TASK_SLEEPING,   // Sleeping (timer-based)
    TASK_ZOMBIE      // Exited, awaiting cleanup
};

// Get task state (atomic)
enum task_state task_get_state(struct task *task);

// Set task state (atomic, internal use)
bool task_set_state(struct task *task,
                    enum task_state old_state,
                    enum task_state new_state);
```

Run-to-Completion Fibers

Fibers are lightweight tasks that run to completion without preemption.

```
// Create a fiber (lightweight task)
struct task *fiber_create(
    fiber_entry_t entry_point,
    void *arg,
    size_t stack_size
);

// Fiber entry point signature
typedef void (*fiber_entry_t)(void *arg);

// Fiber runs to completion, then automatically exits
```

Usage Example:

```
// Fiber entry point (runs to completion)
void my_fiber(void *arg) {
    // Do work without preemption
    compute_result(arg);

    // Fiber exits automatically when function returns
}

// Create and start fiber
void start_fiber(void) {
    void *data = prepare_data();

    struct task *fiber = fiber_create(
        my_fiber,
        data,
        4096 // 4KB stack
    );

    // Fiber starts immediately and runs to completion
}
```

SMP API

Per-CPU Run Queues

```
#include "smp.h"

// Initialize SMP scheduler
void smp_scheduler_init(void);

// Get current CPU's run queue
struct cpu_runqueue *get_current_runqueue(void);

// Get run queue for specific CPU
struct cpu_runqueue *get_cpu_runqueue(uint32_t cpu_id);
```

Task Enqueue/Dequeue

```
// Enqueue task to run queue (lock-free)
int runqueue_enqueue(struct cpu_runqueue *rq, struct task *task);

// Dequeue task from run queue (lock-free)
struct task *runqueue_dequeue(struct cpu_runqueue *rq);

// Peek at next task without dequeuing
struct task *runqueue_peek(struct cpu_runqueue *rq);

// Check if run queue is empty
bool runqueue_is_empty(struct cpu_runqueue *rq);

// Get number of tasks in run queue
uint32_t runqueue_size(struct cpu_runqueue *rq);
```

Usage Example:

```

// Add task to current CPU's run queue
void schedule_task_local(struct task *task) {
    struct cpu_runqueue *rq = get_current_runqueue();

    if (runqueue_enqueue(rq, task) == 0) {
        // Successfully enqueued
        atomic_fetch_add(&task->state, 1);
    }
}

// Get next task to run
struct task *get_next_task(void) {
    struct cpu_runqueue *rq = get_current_runqueue();

    struct task *task = runqueue_dequeue(rq);
    if (task) {
        return task;
    }

    // Try work stealing if local queue empty
    return steal_task();
}

```

Work Stealing

```

// Steal task from another CPU's run queue
struct task *steal_task(void);

// Steal from specific CPU
struct task *steal_task_from_cpu(uint32_t cpu_id);

// Configure work stealing parameters
void set_steal_threshold(uint32_t threshold); // Min tasks before stealing
void set_steal_count(uint32_t count);        // Number of tasks to steal

```

Work Stealing Algorithm:

```

struct task *steal_task(void) {
    uint32_t current_cpu = get_current_cpu_id();
    uint32_t num_cpus = get_num_cpus();

    // Try to steal from each CPU
    for (uint32_t i = 1; i < num_cpus; i++) {
        uint32_t victim_cpu = (current_cpu + i) % num_cpus;

        // Prefer same NUMA node
        if (get_cpu_numa_node(victim_cpu) == get_current_numa_node()) {
            struct task *task = steal_task_from_cpu(victim_cpu);
            if (task) {
                return task;
            }
        }

    }

    // Try other NUMA nodes if necessary
    for (uint32_t i = 1; i < num_cpus; i++) {
        uint32_t victim_cpu = (current_cpu + i) % num_cpus;

        if (get_cpu_numa_node(victim_cpu) != get_current_numa_node()) {
            struct task *task = steal_task_from_cpu(victim_cpu);
            if (task) {
                return task;
            }
        }
    }

    return NULL; // No tasks to steal
}

```

CPU Affinity

```

// Set task CPU affinity
int task_set_affinity(struct task *task, uint32_t cpu_mask);

// Get task CPU affinity
uint32_t task_get_affinity(struct task *task);

// Pin task to specific CPU
int task_pin_to_cpu(struct task *task, uint32_t cpu_id);

// Pin task to NUMA node
int task_pin_to_numa_node(struct task *task, uint32_t node_id);

```

Usage Example:

```
// Pin task to CPU 0
void pin_task_to_cpu0(struct task *task) {
    task_pin_to_cpu(task, 0);
}

// Pin task to NUMA node 1
void pin_task_to_numa1(struct task *task) {
    task_pin_to_numa_node(task, 1);
}

// Allow task to run on CPUs 0-3
void set_task_affinity_0_3(struct task *task) {
    uint32_t mask = 0x0F; // Binary: 00001111 (CPUs 0-3)
    task_set_affinity(task, mask);
}
```

IPI API

Sending IPIs

```
#include "ipi.h"

// IPI types
enum ipi_type {
    IPI_RESCHEDULE, // Force reschedule on target CPU
    IPI_WAKEUP,     // Wake up specific task
    IPI_TLB_FLUSH,  // Flush TLB entries
    IPI_STOP        // Stop CPU
};

// Send IPI to specific CPU
void send_ipi(uint32_t cpu_id, enum ipi_type type, void *data);

// Send IPI to all CPUs except current
void send_ipi_all_but_self(enum ipi_type type, void *data);

// Send IPI to all CPUs
void send_ipi_all(enum ipi_type type, void *data);
```

Usage Example:

```
// Wake up task on remote CPU
void wakeup_task_on_cpu(struct task *task, uint32_t cpu_id) {
    // Prepare IPI data
    struct ipi_wakeup_data data = {
        .task = task
    };

    // Send IPI to target CPU
    send_ipi(cpu_id, IPI_WAKEUP, &data);
}

// Force reschedule on all CPUs
void reschedule_all_cpus(void) {
    send_ipi_all_but_self(IPI_RESCHEDULE, NULL);
}
```

IPI Batching

For efficiency, batch multiple IPIs together:

```
// Start IPI batch
void ipi_batch_start(void);

// Add IPI to batch
void ipi_batch_add(uint32_t cpu_id, enum ipi_type type, void *data);

// Send all batched IPIs
void ipi_batch_send(void);
```

Usage Example:

```
// Wake up multiple tasks efficiently
void wakeup_multiple_tasks(struct task **tasks, uint32_t count) {
    ipi_batch_start();

    for (uint32_t i = 0; i < count; i++) {
        struct task *task = tasks[i];
        uint32_t cpu_id = task_get_cpu(task);

        struct ipi_wakeup_data data = { .task = task };
        ipi_batch_add(cpu_id, IPI_WAKEUP, &data);
    }

    ipi_batch_send();
}
```

Integration Examples

Example 1: Simple Task Creation and Execution

```
#include "scheduler.h"
#include "task.h"

void worker_task(void *arg) {
    int id = *(int *)arg;

    for (int i = 0; i < 100; i++) {
        printf("Worker %d: iteration %d\n", id, i);
        task_yield(); // Yield to other tasks
    }
}

void create_workers(void) {
    for (int i = 0; i < 4; i++) {
        int *id = malloc(sizeof(int));
        *id = i;

        struct task *task = task_create(
            "worker",
            worker_task,
            id,
            PRIORITY_NORMAL,
            8192
        );

        task_start(task);
    }
}
```

Example 2: Producer-Consumer with Fibers

```
#include "task.h"
#include "smp.h"

struct shared_queue {
    _Atomic uint32_t head;
    _Atomic uint32_t tail;
    void *items[256];
};

void producer_fiber(void *arg) {
    struct shared_queue *queue = arg;

    for (int i = 0; i < 1000; i++) {
        void *item = produce_item(i);

        // Enqueue item (lock-free)
        uint32_t tail = atomic_load_explicit(&queue->tail,
                                              memory_order_acquire);
        queue->items[tail % 256] = item;
        atomic_store_explicit(&queue->tail, tail + 1,
                              memory_order_release);
    }
}

void consumer_fiber(void *arg) {
    struct shared_queue *queue = arg;

    for (int i = 0; i < 1000; i++) {
        // Dequeue item (lock-free)
        uint32_t head = atomic_load_explicit(&queue->head,
                                              memory_order_acquire);
        void *item = queue->items[head % 256];
        atomic_store_explicit(&queue->head, head + 1,
                              memory_order_release);

        consume_item(item);
    }
}

void start_producer_consumer(void) {
    struct shared_queue *queue = malloc(sizeof(*queue));
    atomic_init(&queue->head, 0);
    atomic_init(&queue->tail, 0);

    fiber_create(producer_fiber, queue, 4096);
    fiber_create(consumer_fiber, queue, 4096);
}
```

Example 3: NUMA-Aware Task Placement

```
#include "task.h"
#include "smp.h"
#include "memory.h"

void numa_aware_task(void *arg) {
    // Allocate memory on local NUMA node
    void *data = numa_alloc_local(1024 * 1024); // 1MB

    // Process data (stays on local NUMA node)
    process_data(data);

    numa_free(data, 1024 * 1024);
}

void create_numa_aware_tasks(void) {
    uint32_t num_nodes = get_num_numa_nodes();

    for (uint32_t node = 0; node < num_nodes; node++) {
        struct task *task = task_create(
            "numa_task",
            numa_aware_task,
            NULL,
            PRIORITY_NORMAL,
            8192
        );

        // Pin task to NUMA node
        task_pin_to_numa_node(task, node);
        task_start(task);
    }
}
```

Performance Tuning

1. Run Queue Size

Adjust run queue size based on workload:

```
// In smp.h
#define RUNQUEUE_SIZE 256 // Default

// For high task count workloads
#define RUNQUEUE_SIZE 1024

// For low task count workloads
#define RUNQUEUE_SIZE 64
```

2. Work Stealing Threshold

Configure when work stealing occurs:

```
// Steal when victim has at least this many tasks
set_steal_threshold(4);

// Steal this many tasks at once
set_steal_count(2);
```

3. Time Slice

Adjust task time slice for responsiveness vs. throughput:

```
// In scheduler.h
#define TIME_SLICE_MS 10 // Default: 10ms

// For interactive workloads (lower latency)
#define TIME_SLICE_MS 5

// For throughput workloads (fewer context switches)
#define TIME_SLICE_MS 20
```

4. Tickless Mode

Configure tickless scheduling:

```
// Enable tickless mode
scheduler_set_tickless(true);

// Set minimum tick interval (microseconds)
scheduler_set_min_tick_interval(1000); // 1ms

// Set maximum tick interval (microseconds)
scheduler_set_max_tick_interval(100000); // 100ms
```

Troubleshooting

Issue: High Context Switch Rate

Symptoms: High CPU usage, low throughput

Causes:

- Time slice too short
- Too many tasks
- Excessive voluntary yields

Solutions:

```
// Increase time slice
#define TIME_SLICE_MS 20

// Reduce number of tasks
// Batch work instead of creating many small tasks

// Reduce voluntary yields
// Only yield when truly necessary
```

Issue: Load Imbalance

Symptoms: Some CPUs idle while others busy

Causes:

- Work stealing not aggressive enough
- CPU affinity too restrictive
- NUMA placement issues

Solutions:

```
// Lower steal threshold
set_steal_threshold(2);

// Increase steal count
set_steal_count(4);

// Review CPU affinity settings
// Ensure tasks can migrate between CPUs

// Check NUMA placement
// Ensure tasks are on correct NUMA node
```

Issue: High IPI Overhead

Symptoms: High interrupt rate, poor scalability

Causes:

- Too many remote wakeups
- Inefficient IPI batching
- Excessive rescheduling

Solutions:

```
// Use IPI batching
ipi_batch_start();
// ... add multiple IPIs ...
ipi_batch_send();

// Reduce remote wakeups
// Pin tasks to specific CPUs when possible

// Increase time slice to reduce reschedules
#define TIME_SLICE_MS 15
```

Issue: Poor NUMA Performance

Symptoms: High memory latency, low bandwidth

Causes:

- Tasks accessing remote NUMA memory
- Incorrect NUMA node placement
- Work stealing across NUMA nodes

Solutions:

```
// Pin tasks to NUMA nodes
task_pin_to_numa_node(task, node_id);

// Allocate memory on local NUMA node
void *data = numa_alloc_local(size);

// Prefer stealing from same NUMA node
// (already implemented in steal_task())

// Check NUMA topology
uint32_t node = get_cpu_numa_node(cpu_id);
```

Best Practices

1. Use Fibers for Short-Lived Tasks

Fibers are more efficient than full tasks for short-lived work:

```
// Good: Use fiber for quick computation
fiber_create(quick_computation, data, 4096);

// Bad: Use full task for quick computation
task_create("quick", quick_computation, data, PRIORITY_NORMAL, 8192);
```

2. Minimize Voluntary Yields

Only yield when necessary:

```
// Good: Yield after significant work
for (int i = 0; i < 1000; i++) {
    process_item(i);
}
task_yield();

// Bad: Yield too frequently
for (int i = 0; i < 1000; i++) {
    process_item(i);
    task_yield(); // Too frequent!
}
```

3. Use Atomic Operations Correctly

Choose appropriate memory ordering:

```
// Good: Relaxed for counters
atomic_fetch_add_explicit(&counter, 1, memory_order_relaxed);

// Good: Acquire/Release for synchronization
atomic_store_explicit(&flag, 1, memory_order_release);
if (atomic_load_explicit(&flag, memory_order_acquire)) {
    // Synchronized access
}

// Bad: Sequential consistency everywhere (too slow)
atomic_fetch_add(&counter, 1); // Defaults to seq_cst
```

4. Pin Tasks Appropriately

Use CPU affinity for performance-critical tasks:

```
// Good: Pin I/O task to specific CPU
task_pin_to_cpu(io_task, 0);

// Good: Pin compute task to NUMA node
task_pin_to_numa_node(compute_task, node_id);

// Bad: Pin all tasks (prevents load balancing)
for (all tasks) {
    task_pin_to_cpu(task, 0); // All on CPU 0!
}
```

5. Batch Operations

Batch operations for efficiency:

```
// Good: Batch IPI sends
ipi_batch_start();
for (int i = 0; i < count; i++) {
    ipi_batch_add(cpu_id, IPI_WAKEUP, &data[i]);
}
ipi_batch_send();

// Bad: Send IPIs individually
for (int i = 0; i < count; i++) {
    send_ipi(cpu_id, IPI_WAKEUP, &data[i]);
}
```

Conclusion

Phase 3 provides a comprehensive, high-performance scheduler with lock-free concurrency primitives. By following the guidelines in this document, you can effectively use the scheduler API to build efficient, scalable applications on the BDI kernel.

Key Takeaways:

- Use fibers for short-lived tasks
- Minimize voluntary yields
- Choose appropriate memory ordering
- Pin tasks for NUMA awareness
- Batch operations for efficiency
- Monitor and tune performance

For more information, see `PHASE3_ARCHITECTURE.md` for design details and architectural decisions.